

TUGAS BESAR IF2224

MILESTONE 3

SEMANTIC ANALYSIS



Kelompok std_abs

Ega Luthfi Rais	13524115
Salman Faiz Assidqi	13524134
Ahmad Fauzan Putra	13524141
Leonardus Brain Fatolosja	13524146

Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI	2
BAB 1 LANDASAN TEORI	3
1.1 Parse Tree, Abstract Syntax Tree (AST), dan Decorated AST	3
1.2 Attributed Grammar	3
1.3 Symbol Table	4
1.4 Lexical Level dan Display Vector	5
1.5 Type Compatibility dan Implicit Conversion	5
1.6 Klasifikasi Operator dan Aturan Tipe Hasil	6
BAB 2 PERANCANGAN & IMPLEMENTASI	7
2.1 Rancangan Semantic Action, Type Compatibility Rule, dan Predefined Identifier.	7
2.2 Struktur Direktori Modul	16
2.3 Alur Kerja Program	17
2.4 Konversi Parse Tree menjadi AST	18
2.5 Manajemen Scope dan Pendaftaran Identifier	18
2.6 Implementasi visitor: Visit Functions	19
2.7 Type Checking pada Operator	20
2.8 Format Output	20
BAB 3 PENGUJIAN	21
3.1 Pengujian 1: input-1.txt	21
3.2 Pengujian 2: input-2.txt	23
3.3 Pengujian 3: input-3.txt	27
3.4 Pengujian 4: input-4.txt	29
3.5 Pengujian 5: input-5.txt	32
BAB 4 KESIMPULAN DAN SARAN	37
4.1 Kesimpulan	37
4.2 Saran	37
LAMPIRAN	38
REFERENSI	39

BAB 1 LANDASAN TEORI

1.1 Parse Tree, Abstract Syntax Tree (AST), dan Decorated AST

Parse tree adalah representasi pohon yang merekam setiap langkah penurunan suatu kalimat sumber terhadap context-free grammar yang mendefinisikan bahasa. Setiap node internal berkorespondensi langsung dengan satu produksi grammar yang digunakan dan daun-daunnya adalah simbol terminal (token) hasil keluaran lexer.

Abstract Syntax Tree adalah penyederhanaan parse tree yang membuang detail-detail sintaktis yang tidak berperan dalam analisis makna, sambil mempertahankan struktur logis program. Yang biasanya dihilangkan adalah Tanda baca pemisah: titik koma, koma, kurung, titik (kecuali untuk akses field), Keyword yang murni penanda struktur: THEN, DO, BEGIN, END, OF, dan Rantai nonterminal perantara yang hanya berfungsi mengkodekan presedensi operator. Yang biasanya dipertahankan adalah Operator sebagai label node internal (+, *, :=, if, while), Operand sebagai anak-anak node operator, dengan urutan dan kedalaman yang mencerminkan presedensi dan asosiativitas, dan Identifier dan literal sebagai node daun.

Decorated AST (atau Annotated AST) adalah AST yang setiap nodenya telah diperjelas dengan atribut-atribut hasil *semantic analysis*. Atribut yang dihasilkan meliputi:

- type, kode tipe hasil ekspresi (TYPE_INTEGER, TYPE_REAL, TYPE_BOOLEAN, TYPE_CHAR, TYPE_STRING, TYPE_ARRAY, TYPE_RECORD, TYPE_ENUM, TYPE_SUBRANGE, TYPE_VOID, atau TYPE_ERROR ketika type-checking gagal).
- tabIndex, referensi ke entri yang bersangkutan pada tabel identifier (tab); -1 jika node bukan referensi identifier.
- lev, lexical level di mana identifier tersebut dideklarasikan; level 0 adalah scope global tempat predefined identifier berada.
- isLValue, bernilai true jika node merepresentasikan addressable storage yang sah menjadi target assignment, dan false untuk literal serta hasil ekspresi sementara.

1.2 Attributed Grammar

Attributed Grammar adalah perluasan Context-Free Grammar yang menambahkan dua komponen pada setiap simbol non-terminal: (1) himpunan atribut yang membawa informasi semantik, dan (2) semantic rule yang menentukan cara menghitung nilai atribut berdasarkan atribut node tetangga. Atribut dibedakan menjadi:

Jenis Atribut	Arah Aliran	Contoh pada Proyek
Synthesized	Bottom-up, dari anak ke induk	Tipe hasil ekspresi BinOpNode dihitung dari tipe dua operand-nya.

Inherited	Top-down, dari induk/saudara ke anak	Tipe target assignment menentukan kompatibilitas value yang diizinkan; loop variable pada <code>for</code> menentukan tipe yang diharapkan dari <code>start/end</code> .
-----------	--------------------------------------	--

1.3 Symbol Table

Symbol Table merupakan struktur data utama dalam Semantic Analysis. Pada implementasi ini (`symbolTable.hpp`), symbol table disusun atas tiga tabel terpisah:

Tabel	Field	Keterangan
<code>tab</code>	<code>id, link, obj, type, ref, nrm, lev, adr</code>	Menyimpan informasi identifier (konstanta, variabel, tipe, prosedur, fungsi, program, keyword). <code>link</code> mengarah ke identifier sebelumnya dalam scope yang sama, membentuk linked-list per blok.
<code>btab</code>	<code>last, lpar, psze, vsze</code>	Menyimpan informasi blok (program, prosedur, fungsi, record). <code>last</code> menunjuk identifier terakhir blok pada <code>tab</code> ; <code>lpar</code> parameter terakhir; <code>psze</code> ukuran area parameter; <code>vsze</code> ukuran area variabel lokal.
<code>atab</code>	<code>xtyp, etyp, eref, low, high, elsz, size</code>	Menyimpan informasi array atau representasi batas subrange: tipe indeks, tipe elemen, referensi tipe komposit elemen, batas low/high, ukuran elemen, dan ukuran total.

Selain ketiga tabel di atas, `SymbolTable` juga mengelola `display`: array satu dimensi yang memetakan setiap lexical level ke indeks `btab` blok yang sedang aktif pada level tersebut.

Ukuran maksimum diatur melalui konstanta `MAX_TAB_SIZE`, `MAX_BTAB_SIZE`, `MAX_ATAB_SIZE`, `MAX_LEVEL`, dan `MAX_DISPLAY` pada `symbolTable.hpp`.

1.4 Lexical Level dan Display Vector

Lexical level mencerminkan kedalaman nesting blok. Level 0 menampung predefined identifier dan reserved keyword; level 1 menampung deklarasi program; setiap blok prosedur, fungsi, atau record yang nested menambah satu level. Untuk mempercepat lookup identifier dari scope terdalam menuju scope terluar, diterapkan display vector: indeks `display[L]` menunjuk pada btab blok aktif pada level `L`. Lookup bekerja dengan menelusuri linked list `link` pada `tab`, mulai dari `btab[display[L]].last`, untuk setiap level `L` dari aktif menuju 0.

1.5 Type Compatibility dan Implicit Conversion

Type compatibility mendefinisikan kapan dua tipe data boleh berinteraksi (assign, banding, dipakai oleh operator yang sama). Tipe identik selalu kompatibel.

- Integer kompatibel dengan Real (numeric compatibility), promosi Integer $\rightarrow$ Real dilakukan secara implisit.
- Pada assignment, target Real dapat menerima value Integer; sebaliknya, target Integer TIDAK dapat menerima value Real (tidak ada implicit narrowing).
- `TYPE_ERROR` bersifat absorbing: jika salah satu operand sudah `TYPE_ERROR`, hasil ekspresi diset `TYPE_ERROR` untuk mencegah cascade-error report.

Kondisi	Hasil
Tipe identik	Selalu compatible.
Integer $\leftrightarrow$ Real	Compatible (numeric compatibility). Promosi Integer $\rightarrow$ Real dilakukan secara implisit.
Assignment Real := Integer	Diterima sebagai konversi numerik.
Assignment Integer := Real	Pada aturan klasik tidak diizinkan (implicit narrowing dilarang); pada implementasi saat ini diterima karena <code>isCompatible</code> bersifat simetris — lihat catatan implementasi di Bagian 2.2.
Salah satu operand <code>TYPE_ERROR</code>	Dianggap compatible secara internal agar tidak menghasilkan cascade-error.
Subrange $\leftrightarrow$ ordinal lain (Integer, Char, Boolean, Subrange)	Compatible.
Kombinasi lain	Tidak compatible.

1.6 Klasifikasi Operator dan Aturan Tipe Hasil

Bahasa Arion membedakan operator berdasarkan kategori berikut. Implementasi pada `TypeChecker::getResultType` memetakan operator ke kategori, memvalidasi operand, dan menentukan tipe hasil.

Kategori	Operator	Operand Valid	Tipe Hasil
Aritmetika umum	+, -, *	Integer atau Real	Real bila salah satu operand Real, selain itu Integer.
Pembagian real	/	Integer atau Real	Real.
Pembagian bulat	div, mod	Integer	Integer.
Logika biner	and, or	Boolean	Boolean.
Unary numeric	Unary +, unary -	Integer atau Real	Sama dengan tipe operand.
Unary logika	not	Boolean	Boolean.
Equality	==, <>	Tipe compatible (termasuk Boolean)	Boolean.
Order	<, <=, >, >=	Tipe non-Boolean yang compatible; ditolak untuk Array dan Record	Boolean.

`TYPE_ERROR` bersifat absorbing: jika salah satu operand sudah `TYPE_ERROR`, hasil ekspresi diset `TYPE_ERROR` untuk mencegah cascade-error report. Mekanisme ini memungkinkan satu pesan error ditampilkan untuk satu kesalahan, tanpa memunculkan pesan tambahan pada ekspresi induk.

BAB 2 PERANCANGAN & IMPLEMENTASI

Modul Semantic Analysis dirancang sebagai tiga tahap: (1) konversi Parse Tree menjadi AST yang ringkas, (2) inisialisasi Symbol Table dengan predefined identifiers, dan (3) traversal DFS pada AST untuk dekorasi tipe, scope, dan tab index.

2.1 Rancangan Semantic Action, Type Compatibility Rule, dan Predefined Identifier.

A. Semantic Action

Semantic analysis pada program dilakukan dalam dua tahap. Tahap pertama mengubah parse tree menjadi Abstract Syntax Tree (AST) menggunakan syntax-directed translation. Tahap kedua menelusuri AST secara depth-first search (DFS) untuk menghasilkan decorated AST, mengisi symbol table, menghitung tipe, dan melaporkan semantic error.

Konversi Parse Tree Menjadi AST

Pada tahap ini, informasi sintaksis yang tidak lagi diperlukan, seperti terminal `begin`, `end`, `semicolon`, `colon`, dan tanda kurung, tidak dipertahankan sebagai node utama AST. AST hanya mempertahankan struktur yang diperlukan untuk analisis makna program.

Production / Struktur Parse Tree	Semantic Action yang Dilakukan	Node AST yang Dihasilkan
<code><program></code>	Mengambil nama program dari <code><Program-header></code> , mengonversi bagian deklarasi dan compound statement.	<code>ProgramNode (name, declarations, body)</code>
<code><declaration-part></code>	Mengonversi seluruh deklarasi konstanta, tipe, variabel, prosedur, dan fungsi.	<code>DeclarationsNode</code> yang berisi <code>DeclNode</code>
<code><const-declaration></code>	Untuk setiap identifier, mengonversi constant initializer.	<code>ConstDeclNode (name, value)</code>
<code><type-declaration></code>	Untuk setiap identifier, mengonversi definisi tipe.	<code>TypeDeclNode (name, typeDef)</code>
<code><var-declaration></code>	Untuk setiap identifier pada identifier list, membuat	<code>VarDeclNode (name, varType)</code>

	deklarasi dengan tipe hasil konversi.	
<procedure-declaration> / <function-declaration>	Mengambil nama, parameter, return type fungsi, deklarasi lokal, dan body subprogram.	SubprogramDeclNode
<type> berupa identifier	Menyimpan nama primitive type atau named type.	SimpleTypeNode (name)
<array-type>	Mengonversi tipe indeks dan tipe elemen.	ArrayTypeNode (indexType, elementType)
<range>	Mengonversi lower bound dan upper bound.	RangeTypeNode (low, high)
<enumerated>	Menyimpan daftar identifier anggota enumerasi.	EnumeratedTypeNode (values)
<record-type>	Mengonversi seluruh field dan tipe masing-masing field.	RecordTypeNode (fields)
<compound-statement> / <statement-list>	Mengumpulkan statement yang mempunyai makna eksekusi.	BlockNode (statements)
<assignment-statement>	Mengonversi variabel target dan ekspresi nilai.	AssignNode (target, value)
<if-statement>	Mengonversi kondisi, cabang then, dan optional cabang else.	IfNode (condition, thenBranch, elseBranch)
<while-statement>	Mengonversi kondisi dan body.	WhileNode (condition, body)
<repeat-statement>	Mengonversi statement-list sebagai body dan ekspresi until.	RepeatNode (body, condition)
<for-statement>	Mengambil loop variable, nilai awal, nilai akhir, arah to/downto, dan body.	ForNode (varName, start, end, body)
<case-statement>	Mengonversi expression pembanding dan seluruh case block.	CaseNode dan CaseBranchNode
<procedure-function-call>	Mengambil nama subprogram dan daftar argumen.	ProcCallNode (name, args)

<expression> dengan relational operator	Menggabungkan dua simple expression dan operator relasional.	BinOpNode(op, left, right)
<simple-expression> dengan additive operator	Menggabungkan operand secara left-associative.	BinOpNode(op, left, right)
<term> dengan multiplicative operator	Menggabungkan factor secara left-associative.	BinOpNode(op, left, right)
Unary minus atau notsy	Membungkus operand dengan operator unary.	UnaryOpNode(op, operand)
Literal intcon, realcon, charcon, string, true, false	Mengubah terminal menjadi nilai AST.	NumberNode, RealNode, CharNode, StringNode, BoolNode
<variable>	Mengambil nama dasar dan component access berupa indeks array atau field record.	VarNode dan VarComponentNode

Contoh pembentukan AST untuk statement:

```
x := a + 5;
```

Parse tree statement tersebut diringkas menjadi:

```
AssignNode
|-- VarNode("x")
`-- BinOpNode("+")
    |-- VarNode("a")
    `-- NumberNode(5)
```

Dekorasi AST dan Semantic Checking

Setelah AST dibentuk, fungsi `visit(astRoot)` melakukan traversal DFS. Setiap visitor dapat mengunjungi child node, melakukan lookup atau registrasi simbol, menghitung atribut tipe, menambahkan anotasi ke AST, dan melaporkan kesalahan melalui `typeChecker.reportError(...)`.

Node AST / Visitor	Semantic Action Dekorasi
<code>visitProgram</code>	Mendaftarkan nama program sebagai <code>OBJ_PROGRAM</code> pada lexical level 0, membuka scope program, mengunjungi deklarasi dan body, menyimpan referensi block program, lalu menutup scope.
<code>visitDeclarations</code>	Mengunjungi setiap deklarasi sesuai urutan kemunculannya.
<code>visitVarDecl</code>	Memeriksa duplicate identifier pada scope saat ini, menyelesaikan tipe variabel, menghitung offset dan ukuran, memasukkan

	variabel ke <code>tab</code> , lalu mengisi <code>tabIndex</code> , <code>type</code> , dan <code>lev</code> pada node.
<code>visitConstDecl</code>	Memeriksa duplikasi, mengevaluasi literal atau referensi konstanta, termasuk unary initializer seperti <code>B == -A</code> , lalu memasukkan nilai dan tipenya ke <code>tab</code> .
<code>visitTypeDecl</code>	Menyelesaikan simple type atau structured type dan mendaftarkan named type pada <code>tab</code> .
<code>visitSubprogramDecl</code>	Mendaftarkan procedure/function, membuat scope baru pada <code>btabs</code> , memasukkan parameter, menyediakan slot hasil untuk function, memproses local declarations dan body, lalu menutup scope.
<code>visitVar</code>	Melakukan lookup identifier dari scope terdalam ke terluar, memberi anotasi <code>tabIndex</code> , <code>type</code> , <code>lev</code> , dan <code>isLValue</code> , serta memvalidasi akses array dan field record.
<code>visitAssign</code>	Memastikan target berupa l-value, memeriksa assignment compatibility, memeriksa panjang string yang diketahui, dan memeriksa nilai konstanta terhadap batas subrange.
<code>visitBinOp</code>	Mengunjungi operand, memvalidasi operator aritmetika, logika, atau relasional, kemudian menentukan tipe hasil operasi.
<code>visitUnaryOp</code>	Memastikan unary +/- diterapkan pada tipe numerik dan <code>not</code> diterapkan pada Boolean.
<code>visitIf, visitWhile, visitRepeat</code>	Mengunjungi komponen statement dan memastikan condition bertipe Boolean.
<code>visitFor</code>	Memastikan loop variable merupakan variabel ordinal yang telah dideklarasikan; memeriksa kompatibilitas start/end value dan batas subrange jika relevan.
<code>visitCase</code>	Memastikan case expression dan label bersifat ordinal, memeriksa kompatibilitas label, duplicate label, serta batas subrange jika relevan.
<code>visitProcCall</code>	Memastikan identifier merupakan procedure/function; memeriksa argument dan

	parameter; untuk <code>read/readln</code> , mewajibkan argumen berupa l-value.
--	--

Anotasi pada Decorated AST

Setiap node AST memiliki atribut umum yang digunakan pada tahap dekorasi:

Atribut	Keterangan
<code>type</code>	Tipe akhir node atau ekspresi, misalnya <code>TYPE_INTEGER</code> , <code>TYPE_BOOLEAN</code> , atau <code>TYPE_ERROR</code> .
<code>tabIndex</code>	Indeks entri identifier pada tabel <code>tab</code> apabila node berhubungan dengan simbol.
<code>lev</code>	Lexical level tempat identifier dideklarasikan atau ditemukan.
<code>isLValue</code>	Menandai apakah node dapat menjadi target assignment atau input <code>read/readln</code> .
<code>line</code>	Posisi node pada source code untuk membantu pelaporan error.

Semantic error yang ditemukan tidak langsung menghentikan keseluruhan traversal. Error disimpan oleh `TypeChecker` dan dicetak pada akhir semantic analysis bersama decorated AST dan symbol table.

B. Predefined Identifier

Sebelum AST program pengguna didekorasi, `SymbolTable` membuat block awal `btab[0]`, mengaktifkan `display[0]`, dan memasukkan predefined identifier pada lexical level 0.

Predefined Type

Identifier	Object Kind	Type	Keterangan
<code>integer</code>	<code>OBJ_TYPE</code>	<code>TYPE_INTEGER</code>	Tipe bilangan bulat.
<code>real</code>	<code>OBJ_TYPE</code>	<code>TYPE_REAL</code>	Tipe bilangan riil.
<code>boolean</code>	<code>OBJ_TYPE</code>	<code>TYPE_BOOLEAN</code>	Tipe logika.
<code>char</code>	<code>OBJ_TYPE</code>	<code>TYPE_CHAR</code>	Tipe karakter tunggal.
<code>string</code>	<code>OBJ_TYPE</code>	<code>TYPE_STRING</code>	Tipe string.

Predefined Constant

Identifier	Object Kind	Type	Nilai yang Disimpan	Keterangan
------------	-------------	------	---------------------	------------

true	OBJ_CONSTANT	TYPE_BOOLEAN	1	Konstanta Boolean bernilai benar.
false	OBJ_CONSTANT	TYPE_BOOLEAN	0	Konstanta Boolean bernilai salah.

Predefined Procedure

Identifier	Object Kind	Return Type	Aturan Semantik
write	OBJ_PROCEDURE	TYPE_VOID	Built-in output dengan jumlah argumen fleksibel.
writeln	OBJ_PROCEDURE	TYPE_VOID	Built-in output dengan jumlah argumen fleksibel.
read	OBJ_PROCEDURE	TYPE_VOID	Built-in input; setiap argumen harus berupa l-value.
readln	OBJ_PROCEDURE	TYPE_VOID	Built-in input; setiap argumen harus berupa l-value.

Reserved Keyword pada Symbol Table

Selain predefined identifier, implementasi memasukkan reserved keyword sebagai entri OBJ_KEYWORD bertipe TYPE_VOID:

and, array, begin, case, const, div, downto, do, else, end, for, function, if, mod, not, of, or, procedure, program, record, repeat, integer, real, boolean, char, string, then, to, type, until, var, while

Identifier integer, real, boolean, char, dan string muncul sebagai keyword sekaligus sebagai predefined type. Entri predefined type-lah yang digunakan pada proses penyelesaian tipe deklarasi.

C. Type Compatibility Rule

Type yang Digunakan

Type	Pembentukan / Penggunaan
TYPE_INTEGER	Literal <code>intcon</code> dan predefined type <code>integer</code> .
TYPE_REAL	Literal <code>realcon</code> dan predefined type <code>real</code> .

TYPE_BOOLEAN	Literal true/false, operator logika, dan hasil operator relasional.
TYPE_CHAR	Literal charcon dan predefined type char.
TYPE_STRING	Literal string dan predefined type string.
TYPE_ARRAY	Array dengan informasi index type dan element type pada atab.
TYPE_RECORD	Record dengan informasi field pada btab.
TYPE_ENUM	Enumerated type; setiap nilai enumerasi diregistrasikan sebagai konstanta.
TYPE_SUBRANGE	Range ordinal yang memiliki lower bound dan upper bound.
TYPE_VOID	Node/procedure yang tidak menghasilkan nilai.
TYPE_ERROR	Penanda node yang telah mengalami kesalahan semantik.

Compatibility Umum

Implementasi fungsi `isCompatible(type1, type2)` menggunakan aturan berikut:

Kondisi	Hasil
Salah satu tipe adalah TYPE_ERROR	Dianggap compatible secara internal agar tidak menghasilkan error berantai.
Kedua kode tipe sama	Compatible.
Kombinasi TYPE_INTEGER dan TYPE_REAL	Compatible.
Salah satu tipe TYPE_SUBRANGE, sedangkan tipe lainnya TYPE_INTEGER, TYPE_CHAR, TYPE_BOOLEAN, atau TYPE_SUBRANGE	Compatible.
Kondisi lain	Tidak compatible.

Assignment Compatibility

Assignment `target := value` diperiksa menggunakan aturan berikut:

Target	Value	Aturan yang Diterapkan
Tipe yang sama	Tipe yang sama	Diterima.
TYPE_REAL	TYPE_INTEGER	Diterima sebagai konversi numerik.

TYPE_SUBRANGE	TYPE_INTEGER, TYPE_CHAR, TYPE_BOOLEAN, atau TYPE_SUBRANGE	Diterima secara tipe; apabila nilai ordinal dapat diketahui saat analisis, nilainya harus berada di dalam batas subrange.
TYPE_STRING	TYPE_STRING	Diterima; apabila panjang kedua operand diketahui secara statis, panjang harus sama.
L-value apa pun	Value yang tidak compatible	Semantic error incompatible types.
Bukan l-value	Nilai apa pun	Semantic error karena target tidak dapat diberi nilai.

Operator Arithmetic, Logical, Relational, dan Unary

Operator	Operand yang Valid	Tipe Hasil
+, -, *	Dua operand numeric: Integer atau Real	Real apabila salah satu operand Real, selain itu Integer
/	Dua operand numeric	Real
div, mod	Dua operand Integer	Integer
and, or	Dua operand Boolean	Boolean
Unary +, unary -	Satu operand numeric	Sama dengan tipe operand
not	Satu operand Boolean	Boolean
=, <> pada Boolean	Dua operand Boolean yang compatible	Boolean
=, <>, <, <=, >, >= pada tipe non-Boolean	Dua operand compatible, kecuali array dan record	Boolean

Aturan tambahan pada relational expression:

Kasus	Pemeriksaan Tambahan
Kedua operand String	Jika panjang keduanya diketahui secara statis, panjang harus sama.
Salah satu operand Subrange dan operand lainnya memiliki nilai ordinal konstan yang diketahui	Nilai diperiksa agar berada dalam batas subrange.

Operand Array atau Record	Operator relasional ditolak.
---------------------------	------------------------------

Structured Type dan Ordinal Type

Type / Struktur	Aturan Semantik
Ordinal type	Integer, Char, Boolean, dan Subrange dikategorikan sebagai ordinal.
Subrange	Bound tidak boleh Real; tipe lower dan upper bound harus sama; lower bound harus kurang dari atau sama dengan upper bound.
Array	Index type tidak boleh Real; tipe setiap index access harus compatible dengan index type; hasil akses mempunyai element type.
Record	Field access hanya valid jika identifier field ditemukan dalam record; hasil akses mempunyai tipe field.
Enumerated	Setiap nama anggota enum dimasukkan sebagai konstanta TYPE_ENUM; nama anggota tidak boleh terduplikasi pada scope saat ini.

Aturan untuk Statement dan Pemanggilan Subprogram

Konstruk	Aturan Type / Semantic Checking
if condition then ...	condition harus bertipe Boolean.
while condition do ...	condition harus bertipe Boolean.
repeat ... until condition	condition harus bertipe Boolean.
for variable := start to/downto finish do ...	Loop variable harus terdeklarasi, berkategori variable, dan bertipe ordinal; start serta finish harus ordinal dan assignment-compatible dengan loop variable.
case expression of ...	Expression dan label harus ordinal; tipe label harus compatible dengan expression; label tidak boleh duplikat; batas subrange diperiksa jika tersedia.
Procedure/function call	Identifier harus berkategori procedure atau function; argument dievaluasi dan dibandingkan dengan parameter subprogram.
read/ readln	Semua argument harus merupakan l-value karena input dituliskan ke lokasi variabel.

<code>write / writeln</code>	Argumen bersifat fleksibel dan tidak wajib l-value.
------------------------------	---

Keterkaitan dengan Symbol Table

Semantic action dan type compatibility rule di atas didukung oleh tiga tabel simbol berikut:

Tabel	Informasi yang Disimpan
<code>tab</code>	Identifier, object kind, type, referensi tipe komposit/blok, parameter mode, lexical level, nilai atau alamat, dan link ke simbol sebelumnya dalam scope.
<code>btab</code>	Informasi block program, subprogram, dan record berupa identifier terakhir, parameter terakhir, ukuran parameter, dan ukuran variabel lokal.
<code>atab</code>	Informasi array atau representasi batas subrange berupa index type, element type, element reference, lower bound, upper bound, element size, dan total size.

Dengan semantic action tersebut, AST hasil konversi berubah menjadi decorated AST yang memuat informasi tipe, referensi symbol table, lexical level, status l-value, dan posisi source code. Setiap ketidaksesuaian yang ditemukan selama traversal dicatat melalui `TypeChecker` dan ditampilkan sebagai semantic error pada keluaran program.

2.2 Struktur Direktori Modul

Modul Semantic Analysis berada di `src/semantic-analysis/` dan terdiri atas sepuluh berkas:

Berkas	Peran
<code>astNode.hpp</code>	Definisi seluruh kelas node AST (ProgramNode, DeclarationsNode, BlockNode, statement node, expression node, literal, dst.) beserta konstanta <code>TYPE_*</code> , <code>OBJ_*</code> , dan <code>PARAM_*</code> .
<code>semanticAnalyzer.hpp</code>	Deklarasi kelas <code>SemanticAnalyzer</code> : entry point <code>analyze</code> , akses ke <code>SymbolTable</code> dan <code>TypeChecker</code> , serta seluruh prototipe fungsi konversi dan visitor.

<code>semanticAnalyzer.cpp</code>	Konstruktor, destruktur, dan implementasi <code>analyze</code> yang menjalankan tahap konversi parse tree menjadi AST kemudian memanggil <code>visit(astRoot)</code> .
<code>semanticAnalyzer_AST.cpp</code>	Implementasi rekursi <code>convertProgram</code> , <code>convertDeclarationPart</code> , <code>convertCompoundStatement</code> , <code>convertExpression</code> , dan seluruh fungsi konversi parse tree $\rightarrow$ AST.
<code>semanticAnalyzer_DecorAST.cpp</code>	Implementasi traversal DFS <code>visit</code> dan seluruh fungsi <code>visit*</code> yang mendekorasi AST, mengisi symbol table, dan memanggil <code>TypeChecker</code> .
<code>semanticAnalyzer_utils.cpp</code>	Fungsi penunjang: <code>resolveType</code> , <code>resolveTypeNode</code> , <code>helper printDecoratedAST</code> , evaluasi konstanta ordinal, ukuran primitif, dan navigasi parse tree.
<code>symbolTable.hpp</code>	Struktur <code>TabEntry</code> , <code>BtabEntry</code> , <code>AtabEntry</code> , dan kelas <code>SymbolTable</code> beserta konstanta ukuran tabel (<code>MAX_TAB_SIZE</code> , <code>MAX_BTAB_SIZE</code> , <code>dst</code>).
<code>symbolTable.cpp</code>	Implementasi pengelolaan scope (<code>pushScope</code> , <code>popScope</code> , <code>display</code>), <code>lookup</code> , <code>lookupInCurrentScope</code> , registrasi predefined identifier, dan pencetakan tabel.
<code>typeChecker.hpp</code>	Deklarasi kelas <code>TypeChecker</code> dan struktur <code>SemanticError</code> .
<code>typeChecker.cpp</code>	Implementasi <code>isCompatible</code> , <code>isAssignmentCompatible</code> , validasi operator aritmetika/logika/relasional/unary, <code>getResultType</code> , serta penyimpanan dan pencetakan <code>SemanticError</code> .

2.3 Alur Kerja Program

Eksekusi mode all (lexical + syntax + semantic) berjalan sebagai berikut:

1. Source code dibaca lexer (milestone 1), menghasilkan `vector<Token>`.
2. `Vector<Token>` dikonsumsi parser recursive-descent (milestone 2), menghasilkan `ParseNode` root yang merepresentasikan parse tree.
3. `SemanticAnalyzer::analyze(parseTree, tokens)` dipanggil. Tahap konversi membangun `ProgramNode` (root AST) dari parse tree melalui rekursi `convertProgram` ke `convertDeclarationPart` ke `convertCompoundStatement`.
4. `SymbolTable` diinisialisasi di konstruktor: `btab[0]` (blok global) dibuat dan diisi dengan predefined identifier.
5. `visit(astRoot)` dipanggil: traversal DFS men-decorate setiap node, mendaftarkan identifier ke `tab/btab/atab`, dan mereport semantic error ke `TypeChecker`.
6. `visitProgram()` mendaftarkan nama program pada symbol table pada lexical level 0, kemudian membuka scope program menggunakan `pushScope()`. Deklarasi dan body utama program kemudian diproses pada scope tersebut.
7. Setiap deklarasi variabel, konstanta, tipe, prosedur, dan fungsi diregistrasikan ke `tab`. Informasi blok prosedur, fungsi, atau record disimpan pada `btab`, sedangkan informasi array dan subrange disimpan pada `atab`.
8. Setiap penggunaan identifier diperiksa melalui lookup dari scope terdalam menuju scope terluar. Node AST kemudian diberi anotasi berupa tipe akhir, indeks symbol table, lexical level, informasi l-value, dan lokasi pada source code.
9. `TypeChecker` memeriksa kompatibilitas assignment, operator, kondisi kontrol, parameter pemanggilan subprogram, batas subrange, serta kesalahan deklarasi atau penggunaan identifier.
10. Terakhir, program akan mencetak token list, parse tree, decorated AST, simbol tabel (`tab, btab, atab`), dan daftar semantic error.

2.4 Konversi Parse Tree menjadi AST

Konversi dilakukan oleh fungsi rekursif `convertProgram` pada `SemanticAnalyzer.cpp`. Pemilihan node anak dilakukan via helper `findChild` yang mencari child dengan label tertentu pada parse tree.

2.5 Manajemen Scope dan Pendaftaran Identifier

Setiap kali blok baru dibuka (`program body, prosedur, fungsi, record`), fungsi `pushScope()` menambahkan satu entri kosong ke `btab` dan memetakan `display[lev+1]` ke indeks tersebut. Identifier yang dideklarasikan di dalam blok tersebut ditambahkan ke `tab` dengan link diisi dari nilai `btab[blokIni].last` sebelumnya, lalu `btab[blokIni].last` diupdate ke indeks `tab` terbaru.

Lookup identifier memulai traversal dari level aktif: untuk tiap level dari dalam ke luar, ambil indeks btab via display, lalu telusuri linked list lewat field link sambil membandingkan nama dengan target. Pencocokan pertama yang ditemukan dikembalikan, mengikuti aturan shadowing nested scope.

lookupInCurrentScope membatasi pencarian hanya pada blok aktif sehingga deteksi duplicate identifier dalam satu blok dapat dilakukan tanpa menyilangi scope luar.

2.6 Implementasi visitor: Visit Functions

Setiap kelas node AST memiliki visit function yang mengimplementasikan semantic rule untuk produksi grammar yang bersangkutan. Ringkasan peran tiap visit function diberikan pada tabel berikut.

Visit Function	Tugas Semantik Utama
visitProgram	Mendaftarkan nama program sebagai OBJ_PROGRAM di level 0, push scope program, kunjungi declarations & body, pop scope.
visitVarDecl	Cek duplicate identifier pada scope aktif, resolveTypeNode, alokasikan adr, daftarkan sebagai OBJ_VARIABLE.
visitConstDecl	Evaluasi initializer (harus literal atau identifier konstan), simpan nilai pada field adr, daftarkan sebagai OBJ_CONSTANT.
visitTypeDecl	Resolve type body, daftarkan sebagai OBJ_TYPE dengan ref menunjuk ke struktur tipe komposit bila ada.
visitSubprogramDecl	Daftarkan prosedur/fungsi pada level enclosing, push scope, daftarkan parameter, kunjungi body, hitung psze & vsze, pop scope.
visitAssign	Visit target (harus lvalue) dan value, validasi assignment compatibility melalui TypeChecker::isAssignmentCompatible.
visitBinOp	Visit left & right, panggil getResultType; jika TYPE_ERROR, laporkan operator tidak terdefinisi untuk pasangan tipe ini.
visitUnaryOp	Visit operand, panggil getUnaryResultType; laporkan jika tidak valid.
visitIf / visitWhile / visitRepeat	Validasi bahwa condition bertipe Boolean; visit thenBranch/elseBranch/body.
visitFor	Validasi loop variable terdaftar, ordinal, dan bertipe Variable; validasi start & end ordinal & kompatibel dengan loop variable.
visitCase	Visit expression; untuk tiap CaseBranch validasi konstanta selektor.
visitProcCall	Resolve callee; cek ia OBJ_PROCEDURE/OBJ_FUNCTION; cek arity dan compatibility tiap argumen.

visitVar	Lookup identifier; tetapkan type, tabIndex, lev, dan isLValue. Untuk array access, validasi index type; untuk record field, validasi field exists.
----------	--

2.7 Type Checking pada Operator

`TypeChecker::getResultType` implementasinya memetakan operator ke kategori (relasional, logika, pembagian real, pembagian bulat, atau aritmatika umum) dan untuk masing-masing kategori memvalidasi operand serta menentukan tipe hasil. Bila operasi tidak valid, dikembalikan `TYPE_ERROR` agar caller dapat mereport kesalahan dengan pesan operator + tidak terdefinisi untuk pasangan tipe ini, sambil tetap menyebarkan `TYPE_ERROR` ke ekspresi induk untuk mencegah pesan duplikat.

2.8 Format Output

printDecoratedAST mencetak AST dengan indentasi dua spasi per level, menampilkan label node beserta atribut hasil dekorasi untuk node referensi identifier dan ekspresi. **printSymbolTable** mencetak tiga tabel berurutan dengan format aligned kolom: `tab (idx, id, obj, type, ref, nrm, lev, adr, link)`, `btabs (idx, last, lpar, psze, vsze)`, dan `atab (idx, xtyp, etyp, eref, low, high, elsz, size)`. **printErrors** mencetak tiap entri `SemanticError` pada baris terpisah.

BAB 3 PENGUJIAN

3.1 Pengujian 1: input-1.txt

Masukan (input)
<pre>program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
Keluaran (output)

```
Pilihan: 4
=== [!] Input: Source Code ===
=== [!] Output: Decorated AST, Symbol Table, Semantic Errors ===
=== Input file berada di folder milestone-1 [test/milestone-1] ===
=== Output file berada di folder milestone-3 [test/milestone-3] ===
File input source: input-1.txt
File output semantic (kosongkan kalau tidak mau print ke file):
```

```
===== Decorated AST =====
```

```
Program: Hello [tab=43] type=void lev=0 line=1:1
├─ Declarations: [2 declarations] line=2:1
│   ├── VarDecl: a [tab=44] type=integer lev=1 line=3:7 <type simple=integer resolved=integer>
│   └── VarDecl: b [tab=45] type=integer lev=1 line=3:7 <type simple=integer resolved=integer>
├─ Block: [4 statements] line=5:1
│   ├── Assign: type=integer line=5:1
│   │   ├── Var: a [tab=44] type=integer lev=1 lval=true line=5:1
│   │   └── Number: 5 type=integer line=5:6
│   ├── Assign: type=integer line=6:1
│   │   ├── Var: b [tab=45] type=integer lev=1 lval=true line=6:1
│   │   ├── BinOp: + type=integer line=6:8
│   │   │   ├── Var: a [tab=44] type=integer lev=1 lval=true line=6:6
│   │   │   └── Number: 10 type=integer line=6:10
│   ├── Call: writeln [tab=39] type=void lev=0 args=2 line=7:1
│   │   ├── String: "Result = " type=string length=9 line=7:9
│   │   └── Var: b [tab=45] type=integer lev=1 lval=true line=7:22
└─ EmptyStmt
```

```
=== Symbol Table (tab) ===
```

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	keyword	void	0	value	0	0	-
1	array	keyword	void	0	value	0	0	-

```
===== Decorated AST =====
```

```
Program: Hello [tab=43] type=void lev=0 line=1:1
├─ Declarations: [2 declarations] line=2:1
│   ├── VarDecl: a [tab=44] type=integer lev=1 line=3:7 <type simple=integer resolved=integer>
│   └── VarDecl: b [tab=45] type=integer lev=1 line=3:7 <type simple=integer resolved=integer>
├─ Block: [4 statements] line=5:1
│   ├── Assign: type=integer line=5:1
│   │   ├── Var: a [tab=44] type=integer lev=1 lval=true line=5:1
│   │   └── Number: 5 type=integer line=5:6
│   ├── Assign: type=integer line=6:1
│   │   ├── Var: b [tab=45] type=integer lev=1 lval=true line=6:1
│   │   ├── BinOp: + type=integer line=6:8
│   │   │   ├── Var: a [tab=44] type=integer lev=1 lval=true line=6:6
│   │   │   └── Number: 10 type=integer line=6:10
│   ├── Call: writeln [tab=39] type=void lev=0 args=2 line=7:1
│   │   ├── String: "Result = " type=string length=9 line=7:9
│   │   └── Var: b [tab=45] type=integer lev=1 lval=true line=7:22
└─ EmptyStmt
```

```
=== Symbol Table (tab) ===
```

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	keyword	void	0	value	0	0	-
1	array	keyword	void	0	value	0	0	-
2	begin	keyword	void	0	value	0	0	-
3	case	keyword	void	0	value	0	0	-
4	const	keyword	void	0	value	0	0	-
5	div	keyword	void	0	value	0	0	-
6	downto	keyword	void	0	value	0	0	-
7	do	keyword	void	0	value	0	0	-
8	else	keyword	void	0	value	0	0	-
9	end	keyword	void	0	value	0	0	-

10	for	keyword	void	0	value	0	0	-
11	function	keyword	void	0	value	0	0	-
12	if	keyword	void	0	value	0	0	-
13	mod	keyword	void	0	value	0	0	-
14	not	keyword	void	0	value	0	0	-
15	of	keyword	void	0	value	0	0	-
16	or	keyword	void	0	value	0	0	-
17	procedure	keyword	void	0	value	0	0	-
18	program	keyword	void	0	value	0	0	-
19	record	keyword	void	0	value	0	0	-
20	repeat	keyword	void	0	value	0	0	-
21	integer	keyword	void	0	value	0	0	-
22	real	keyword	void	0	value	0	0	-
23	boolean	keyword	void	0	value	0	0	-
24	char	keyword	void	0	value	0	0	-
25	string	keyword	void	0	value	0	0	-
26	then	keyword	void	0	value	0	0	-
27	to	keyword	void	0	value	0	0	-
28	type	keyword	void	0	value	0	0	-
29	until	keyword	void	0	value	0	0	-
30	var	keyword	void	0	value	0	0	-
31	while	keyword	void	0	value	0	0	-
32	integer	type	integer	0	value	0	1	-
33	real	type	real	0	value	0	1	32
34	boolean	type	boolean	0	value	0	1	33
35	char	type	char	0	value	0	1	34
36	string	type	string	0	value	0	1	35
37	true	constant	boolean	0	value	0	1	36
38	false	constant	boolean	0	value	0	0	37
39	writeln	procedure	void	0	value	0	0	38
40	readln	procedure	void	0	value	0	0	39
41	write	procedure	void	0	value	0	0	40
42	read	procedure	void	0	value	0	0	41
43	Hello	program	void	1	value	0	0	42
44	a	variable	integer	0	value	1	0	-
45	b	variable	integer	0	value	1	1	44

=== Block Table (btab) ===

idx	last	lpar	psze	vsze
0	43	0	0	0
1	45	-	0	2

=== Array Table (atab) ===

idx	xtyp	etyp	eref	low	high	elsz	size
(empty)							

3.2 Pengujian 2: input-2.txt

Masukan (input)
<pre>program Operators; var x, y: integer; begin x := 10; y := 3; if x >= y then writeln(x div y); if x <> y then writeln(x mod y); if x <= 20 then writeln(x * y - 1); end.</pre> <pre>program Operators; var x, y: integer; begin x := 10; y := 3; if x >= y then writeln(x div y); if x <> y then writeln(x mod y); if x <= 20 then writeln(x * y - 1); end.</pre>
Keluaran (output)

```

=== [!] Input: Source Code ===
=== [!] Output: Decorated AST, Symbol Table, Semantic Errors ===
=== Input file berada di folder milestone-1 [test/milestone-1] ===
=== Output file berada di folder milestone-3 [test/milestone-3] ===
File input source: input-2.txt
File output semantic (kosongkan kalau tidak mau print ke file):

```

===== Decorated AST =====

```

Program: Operators [tab=43] type=void lev=0 line=1:1
├── Declarations: [2 declarations] line=2:1
│   ├── VarDecl: x [tab=44] type=integer lev=1 line=3:7 <type simple=integer resolved=integer>
│   └── VarDecl: y [tab=45] type=integer lev=1 line=3:7 <type simple=integer resolved=integer>
├── Block: [6 statements] line=5:1
│   ├── Assign: type=integer line=5:1
│   │   ├── Var: x [tab=44] type=integer lev=1 lval=true line=5:1
│   │   └── Number: 10 type=integer line=5:6
│   ├── Assign: type=integer line=6:1
│   │   ├── Var: y [tab=45] type=integer lev=1 lval=true line=6:1
│   │   └── Number: 3 type=integer line=6:6
│   ├── If: line=7:1
│   │   ├── Condition:
│   │   │   ├── BinOp: >= type=boolean line=7:4
│   │   │   │   ├── Var: x [tab=44] type=integer lev=1 lval=true line=7:4
│   │   │   │   └── Var: y [tab=45] type=integer lev=1 lval=true line=7:9
│   │   └── Then:
│   │       ├── Call: writeln [tab=39] type=void lev=0 args=1 line=8:3
│   │       └── BinOp: div type=integer line=8:13
│   │           ├── Var: x [tab=44] type=integer lev=1 lval=true line=8:11
│   │           └── Var: y [tab=45] type=integer lev=1 lval=true line=8:17
│   └── If: line=9:1
│       ├── Condition:

```

===== Decorated AST =====

```

Program: Operators [tab=43] type=void lev=0 line=1:1
├── Declarations: [2 declarations] line=2:1
│   ├── VarDecl: x [tab=44] type=integer lev=1 line=3:7 <type simple=integer resolved=integer>
│   └── VarDecl: y [tab=45] type=integer lev=1 line=3:7 <type simple=integer resolved=integer>
├── Block: [6 statements] line=5:1
│   ├── Assign: type=integer line=5:1
│   │   ├── Var: x [tab=44] type=integer lev=1 lval=true line=5:1
│   │   └── Number: 10 type=integer line=5:6
│   ├── Assign: type=integer line=6:1
│   │   ├── Var: y [tab=45] type=integer lev=1 lval=true line=6:1
│   │   └── Number: 3 type=integer line=6:6
│   ├── If: line=7:1
│   │   ├── Condition:
│   │   │   ├── BinOp: >= type=boolean line=7:4
│   │   │   │   ├── Var: x [tab=44] type=integer lev=1 lval=true line=7:4
│   │   │   │   └── Var: y [tab=45] type=integer lev=1 lval=true line=7:9
│   │   └── Then:
│   │       ├── Call: writeln [tab=39] type=void lev=0 args=1 line=8:3
│   │       └── BinOp: div type=integer line=8:13
│   │           ├── Var: x [tab=44] type=integer lev=1 lval=true line=8:11
│   │           └── Var: y [tab=45] type=integer lev=1 lval=true line=8:17
│   └── If: line=9:1
│       ├── Condition:
│       │   ├── BinOp: <> type=boolean line=9:4
│       │   │   ├── Var: x [tab=44] type=integer lev=1 lval=true line=9:4
│       │   │   └── Var: y [tab=45] type=integer lev=1 lval=true line=9:9
│       └── Then:
│           ├── Call: writeln [tab=39] type=void lev=0 args=1 line=10:3
│           └── BinOp: mod type=integer line=10:13
│               └── Var: x [tab=44] type=integer lev=1 lval=true line=10:11

```



```
=== Array Table (atab) ===
idx  xtyp      etyp      eref  low   high  elsz  size
-----
(empty)
```

3.3 Pengujian 3: input-3.txt

Masukan (input)

```
program Comments;
{ ini komentar kurung kurawal }
var
a: integer;
begin
(* ini komentar
  .. bintang kurung *)
a := 42;
{ komentar lagi }
end.
```

```
program Comments;
{ ini komentar kurung kurawal }
var
a: integer;
begin
(* ini komentar
  bintang kurung *)
a := 42;
{ komentar lagi }
end.
```

Keluaran (output)

```

4. semantic
5. semua
Pilihan: 4
=== [!] Input: Source Code ===
=== [!] Output: Decorated AST, Symbol Table, Semantic Errors ===
=== Input file berada di folder milestone-1 [test/milestone-1] ===
=== Output file berada di folder milestone-3 [test/milestone-3] ===
File input source: input-3.txt
File output semantic (kosongkan kalau tidak mau print ke file):

```

```

===== Decorated AST =====
Program: Comments [tab=43] type=void lev=0 line=1:1
├─ Declarations: [1 declarations] line=3:1
│   └─ VarDecl: a [tab=44] type=integer lev=1 line=4:4 <type simple=integer resolved=integer>
├─ Block: [2 statements] line=8:1
│   └─ Assign: type=integer line=8:1
│       └─ Var: a [tab=44] type=integer lev=1 lval=true line=8:1
│           └─ Number: 42 type=integer line=8:6
└─ EmptyStmt

```

```

=== Symbol Table (tab) ===
idx id      obj      type      ref      nrm      lev      adr      link
-----
0  and      keyword    void      0      value    0      0      -
1  array     keyword    void      0      value    0      0      -
2  begin     keyword    void      0      value    0      0      -
3  case      keyword    void      0      value    0      0      -
4  const     keyword    void      0      value    0      0      -
5  div       keyword    void      0      value    0      0      -
6  downto    keyword    void      0      value    0      0      -
7  do        keyword    void      0      value    0      0      -

```

```

===== Decorated AST =====
Program: Comments [tab=43] type=void lev=0 line=1:1
├─ Declarations: [1 declarations] line=3:1
│   └─ VarDecl: a [tab=44] type=integer lev=1 line=4:4 <type simple=integer
resolved=integer>
├─ Block: [2 statements] line=8:1
│   └─ Assign: type=integer line=8:1
│       └─ Var: a [tab=44] type=integer lev=1 lval=true line=8:1
│           └─ Number: 42 type=integer line=8:6
└─ EmptyStmt

```

```

=== Symbol Table (tab) ===
idx id      obj      type      ref      nrm      lev      adr      link
-----
0  and      keyword    void      0      value    0      0      -
1  array     keyword    void      0      value    0      0      -
2  begin     keyword    void      0      value    0      0      -
3  case      keyword    void      0      value    0      0      -
4  const     keyword    void      0      value    0      0      -
5  div       keyword    void      0      value    0      0      -
6  downto    keyword    void      0      value    0      0      -
7  do        keyword    void      0      value    0      0      -
8  else      keyword    void      0      value    0      0      -
9  end       keyword    void      0      value    0      0      -
10 for      keyword    void      0      value    0      0      -
11 function  keyword    void      0      value    0      0      -
12 if       keyword    void      0      value    0      0      -
13 mod      keyword    void      0      value    0      0      -
14 not      keyword    void      0      value    0      0      -
15 of       keyword    void      0      value    0      0      -
16 or       keyword    void      0      value    0      0      -
17 procedure keyword    void      0      value    0      0      -
18 program  keyword    void      0      value    0      0      -
19 record   keyword    void      0      value    0      0      -

```

20	repeat	keyword	void	0	value	0	0	-
21	integer	keyword	void	0	value	0	0	-
22	real	keyword	void	0	value	0	0	-
23	boolean	keyword	void	0	value	0	0	-
24	char	keyword	void	0	value	0	0	-
25	string	keyword	void	0	value	0	0	-
26	then	keyword	void	0	value	0	0	-
27	to	keyword	void	0	value	0	0	-
28	type	keyword	void	0	value	0	0	-
29	until	keyword	void	0	value	0	0	-
30	var	keyword	void	0	value	0	0	-
31	while	keyword	void	0	value	0	0	-
32	integer	type	integer	0	value	0	1	-
33	real	type	real	0	value	0	1	32
34	boolean	type	boolean	0	value	0	1	33
35	char	type	char	0	value	0	1	34
36	string	type	string	0	value	0	1	35
37	true	constant	boolean	0	value	0	1	36
38	false	constant	boolean	0	value	0	0	37
39	writeln	procedure	void	0	value	0	0	38
40	readln	procedure	void	0	value	0	0	39
41	write	procedure	void	0	value	0	0	40
42	read	procedure	void	0	value	0	0	41
43	Comments	program	void	1	value	0	0	42
44	a	variable	integer	0	value	1	0	-

=== Block Table (btab) ===

idx	last	lpar	psze	vsze
0	43	0	0	0
1	44	-	0	1

=== Array Table (atab) ===

idx	xtyp	etyp	eref	low	high	elsz	size
(empty)							

3.4 Pengujian 4: input-4.txt

Masukan (input)

```
program Literals;  
var  
c: char;  
s: string;  
r: real;  
begin  
c := 'A';  
s := 'Hello World';  
r := 3.14;  
writeln(c);  
writeln(s);  
writeln(r + 100.05);  
end.
```

```
program Literals;  
var  
c: char;  
s: string;  
r: real;  
begin  
c := 'A';  
s := 'Hello World';  
r := 3.14;  
writeln(c);  
writeln(s);  
writeln(r + 100.05);  
end.
```

Keluaran (output)

```

=== [!] Input: Source Code ===
=== [!] Output: Decorated AST, Symbol Table, Semantic Errors ===
=== Input file berada di folder milestone-1 [test/milestone-1] ===
=== Output file berada di folder milestone-3 [test/milestone-3] ===
File input source: input-4.txt
File output semantic (kosongkan kalau tidak mau print ke file):

```

===== Decorated AST =====

```

Program: Literals [tab=43] type=void lev=0 line=1:1
├─ Declarations: [3 declarations] line=2:1
│   ├── VarDecl: c [tab=44] type=char lev=1 line=3:4 <type simple=char resolved=char>
│   ├── VarDecl: s [tab=45] type=string lev=1 line=4:4 <type simple=string resolved=string>
│   └── VarDecl: r [tab=46] type=real lev=1 line=5:4 <type simple=real resolved=real>
├─ Block: [7 statements] line=7:1
│   ├── Assign: type=char line=7:1
│   │   ├── Var: c [tab=44] type=char lev=1 lval=true line=7:1
│   │   └── Char: 'A' type=char line=7:6
│   ├── Assign: type=string line=8:1
│   │   ├── Var: s [tab=45] type=string lev=1 lval=true line=8:1
│   │   └── String: "Hello World" type=string length=11 line=8:6
│   ├── Assign: type=real line=9:1
│   │   ├── Var: r [tab=46] type=real lev=1 lval=true line=9:1
│   │   └── Real: 3.14 type=real line=9:6
│   ├── Call: writeln [tab=39] type=void lev=0 args=1 line=10:1
│   │   └── Var: c [tab=44] type=char lev=1 lval=true line=10:9
│   ├── Call: writeln [tab=39] type=void lev=0 args=1 line=11:1
│   │   └── Var: s [tab=45] type=string lev=1 lval=true line=11:9
│   ├── Call: writeln [tab=39] type=void lev=0 args=1 line=12:1
│   │   └── BinOp: + type=real line=12:11
│   │       ├── Var: r [tab=46] type=real lev=1 lval=true line=12:9
│   │       └── Real: 100.05 type=real line=12:13

```

===== Decorated AST =====

```

Program: Literals [tab=43] type=void lev=0 line=1:1
├─ Declarations: [3 declarations] line=2:1
│   ├── VarDecl: c [tab=44] type=char lev=1 line=3:4 <type simple=char resolved=char>
│   ├── VarDecl: s [tab=45] type=string lev=1 line=4:4 <type simple=string resolved=string>
│   └── VarDecl: r [tab=46] type=real lev=1 line=5:4 <type simple=real resolved=real>
├─ Block: [7 statements] line=7:1
│   ├── Assign: type=char line=7:1
│   │   ├── Var: c [tab=44] type=char lev=1 lval=true line=7:1
│   │   └── Char: 'A' type=char line=7:6
│   ├── Assign: type=string line=8:1
│   │   ├── Var: s [tab=45] type=string lev=1 lval=true line=8:1
│   │   └── String: "Hello World" type=string length=11 line=8:6
│   ├── Assign: type=real line=9:1
│   │   ├── Var: r [tab=46] type=real lev=1 lval=true line=9:1
│   │   └── Real: 3.14 type=real line=9:6
│   ├── Call: writeln [tab=39] type=void lev=0 args=1 line=10:1
│   │   └── Var: c [tab=44] type=char lev=1 lval=true line=10:9
│   ├── Call: writeln [tab=39] type=void lev=0 args=1 line=11:1
│   │   └── Var: s [tab=45] type=string lev=1 lval=true line=11:9
│   ├── Call: writeln [tab=39] type=void lev=0 args=1 line=12:1
│   │   └── BinOp: + type=real line=12:11
│   │       ├── Var: r [tab=46] type=real lev=1 lval=true line=12:9
│   │       └── Real: 100.05 type=real line=12:13
└─ EmptyStmt

```

=== Symbol Table (tab) ===

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	keyword	void	0	value	0	0	-
1	array	keyword	void	0	value	0	0	-
2	begin	keyword	void	0	value	0	0	-
3	case	keyword	void	0	value	0	0	-
4	const	keyword	void	0	value	0	0	-

5	div	keyword	void	0	value	0	0	-
6	downto	keyword	void	0	value	0	0	-
7	do	keyword	void	0	value	0	0	-
8	else	keyword	void	0	value	0	0	-
9	end	keyword	void	0	value	0	0	-
10	for	keyword	void	0	value	0	0	-
11	function	keyword	void	0	value	0	0	-
12	if	keyword	void	0	value	0	0	-
13	mod	keyword	void	0	value	0	0	-
14	not	keyword	void	0	value	0	0	-
15	of	keyword	void	0	value	0	0	-
16	or	keyword	void	0	value	0	0	-
17	procedure	keyword	void	0	value	0	0	-
18	program	keyword	void	0	value	0	0	-
19	record	keyword	void	0	value	0	0	-
20	repeat	keyword	void	0	value	0	0	-
21	integer	keyword	void	0	value	0	0	-
22	real	keyword	void	0	value	0	0	-
23	boolean	keyword	void	0	value	0	0	-
24	char	keyword	void	0	value	0	0	-
25	string	keyword	void	0	value	0	0	-
26	then	keyword	void	0	value	0	0	-
27	to	keyword	void	0	value	0	0	-
28	type	keyword	void	0	value	0	0	-
29	until	keyword	void	0	value	0	0	-
30	var	keyword	void	0	value	0	0	-
31	while	keyword	void	0	value	0	0	-
32	integer	type	integer	0	value	0	1	-
33	real	type	real	0	value	0	1	32
34	boolean	type	boolean	0	value	0	1	33
35	char	type	char	0	value	0	1	34
36	string	type	string	0	value	0	1	35
37	true	constant	boolean	0	value	0	1	36
38	false	constant	boolean	0	value	0	0	37
39	writeln	procedure	void	0	value	0	0	38
40	readln	procedure	void	0	value	0	0	39
41	write	procedure	void	0	value	0	0	40
42	read	procedure	void	0	value	0	0	41
43	Literals	program	void	1	value	0	0	42
44	c	variable	char	0	value	1	0	-
45	s	variable	string	0	value	1	1	44
46	r	variable	real	0	value	1	2	45

=== Block Table (btab) ===

idx	last	lpar	psze	vsze
0	43	0	0	0
1	46	-	0	3

=== Array Table (atab) ===

idx	xtyp	etyp	eref	low	high	elsz	size
(empty)							

3.5 Pengujian 5: input-5.txt

Masukan (input)

```
program LoopArray;  
var  
arr: array[1..5] of integer;  
i: integer;  
begin  
for i := 1 to 5 do  
  arr[i] := i * 2;  
i := 1;  
repeat  
  writeln(arr[i]);  
  i := i + 1;  
until i == 5;  
while i > 0 do  
  i := i - 1;  
end.  
|
```

```
program LoopArray;  
var  
arr: array[1..5] of integer;  
i: integer;  
begin  
for i := 1 to 5 do  
  arr[i] := i * 2;  
i := 1;  
repeat  
  writeln(arr[i]);  
  i := i + 1;  
until i == 5;  
while i > 0 do  
  i := i - 1;  
end.
```

Keluaran (output)

```

=== [!] Input: Source Code ===
=== [!] Output: Decorated AST, Symbol Table, Semantic Errors ===
=== Input file berada di folder milestone-1 [test/milestone-1] ===
=== Output file berada di folder milestone-3 [test/milestone-3] ===
File input source: input-5.txt
File output semantic (kosongkan kalau tidak mau print ke file):

```

```

===== Decorated AST =====
Program: LoopArray [tab=43] type=void lev=0 line=1:1
├─ Declarations: [2 declarations] line=2:1
│   ├── VarDecl: arr [tab=44] type=array lev=1 line=3:6 <type array atab=0 size=5>
│   └── VarDecl: i [tab=45] type=integer lev=1 line=4:4 <type simple=integer resolved=integer>
├─ Block: [5 statements] line=6:1
│   ├── For: i to [varTab=45] type=void line=6:1
│   │   ├── Number: 1 type=integer line=6:10
│   │   ├── Number: 5 type=integer line=6:15
│   │   ├── Assign: type=integer line=7:3
│   │   │   ├── Var: arr[i] [tab=44] type=integer lev=1 lval=true line=7:3
│   │   │   ├── BinOp: * type=integer line=7:15
│   │   │   │   ├── Var: i [tab=45] type=integer lev=1 lval=true line=7:13
│   │   │   │   └── Number: 2 type=integer line=7:17
│   │   └── Assign: type=integer line=8:1
│   │       ├── Var: i [tab=45] type=integer lev=1 lval=true line=8:1
│   │       └── Number: 1 type=integer line=8:6
│   └── Repeat: line=9:1
│       └── Body:
│           └── Block: [3 statements] line=10:3
│               ├── Call: writeln [tab=39] type=void lev=0 args=1 line=10:3
│               │   └── Var: arr[i] [tab=44] type=integer lev=1 lval=true line=10:11
│               └── Assign: type=integer line=11:3

```

```

===== Decorated AST =====
Program: LoopArray [tab=43] type=void lev=0 line=1:1
├─ Declarations: [2 declarations] line=2:1
│   ├── VarDecl: arr [tab=44] type=array lev=1 line=3:6 <type array atab=0 size=5>
│   └── VarDecl: i [tab=45] type=integer lev=1 line=4:4 <type simple=integer resolved=integer>
├─ Block: [5 statements] line=6:1
│   ├── For: i to [varTab=45] type=void line=6:1
│   │   ├── Number: 1 type=integer line=6:10
│   │   ├── Number: 5 type=integer line=6:15
│   │   ├── Assign: type=integer line=7:3
│   │   │   ├── Var: arr[i] [tab=44] type=integer lev=1 lval=true line=7:3
│   │   │   ├── BinOp: * type=integer line=7:15
│   │   │   │   ├── Var: i [tab=45] type=integer lev=1 lval=true line=7:13
│   │   │   │   └── Number: 2 type=integer line=7:17
│   │   └── Assign: type=integer line=8:1
│   │       ├── Var: i [tab=45] type=integer lev=1 lval=true line=8:1
│   │       └── Number: 1 type=integer line=8:6
│   └── Repeat: line=9:1
│       └── Body:
│           └── Block: [3 statements] line=10:3
│               ├── Call: writeln [tab=39] type=void lev=0 args=1 line=10:3
│               │   └── Var: arr[i] [tab=44] type=integer lev=1 lval=true line=10:11
│               ├── Assign: type=integer line=11:3
│               │   ├── Var: i [tab=45] type=integer lev=1 lval=true line=11:3
│               │   ├── BinOp: + type=integer line=11:10
│               │   │   ├── Var: i [tab=45] type=integer lev=1 lval=true line=11:8
│               │   │   └── Number: 1 type=integer line=11:12
│               └── EmptyStmt
│           └── Until:
│               ├── BinOp: = type=boolean line=12:7
│               │   ├── Var: i [tab=45] type=integer lev=1 lval=true line=12:7
│               │   └── Number: 5 type=integer line=12:12
│               └── While: line=13:1
│                   └── Condition:

```


BAB 4 KESIMPULAN DAN SARAN

4.1 Kesimpulan

Pada Milestone 3 ini, kelompok kami berhasil mengimplementasikan modul Semantic Analysis untuk bahasa Arion sebagai kelanjutan dari Lexical Analyzer dan Syntax Analyzer. Modul semantic analyzer menerima parse tree hasil parser, kemudian mengubahnya menjadi Abstract Syntax Tree (AST) yang lebih ringkas dan melakukan proses dekorasi AST melalui traversal berbasis visitor. Dekorasi tersebut mencakup anotasi tipe data, lexical level, referensi entri symbol table, serta informasi apakah suatu node dapat menjadi target assignment.

Program juga telah dilengkapi dengan symbol table yang terdiri atas tab, btab, dan atab. Ketiga tabel tersebut digunakan untuk menyimpan identifier, informasi block/scope, serta informasi tipe array dan subrange. Selain itu, program telah menginisialisasi predefined identifier dan reserved keyword, serta mampu melakukan lookup identifier berdasarkan scope aktif menggunakan display vector.

Berdasarkan pengujian yang dilakukan, program telah mampu menangani deklarasi variabel, literal, operasi aritmetika dan relasional, pemanggilan prosedur bawaan, array sederhana, serta struktur perulangan. Keluaran program berupa decorated AST dan symbol table dapat menampilkan hasil analisis semantik secara terstruktur sehingga membantu proses pemeriksaan tipe dan scope.

4.2 Saran

- Mengganti display vector berbasis array fixed-size dengan stack dinamis agar batas tidak menjadi kendala pada program dengan nesting sangat dalam.
- Menambahkan modul code generation sebagai milestone lanjutan, memanfaatkan Decorated AST yang sudah memuat seluruh informasi tipe dan offset.
- Memperkaya pesan error dengan source-line context selain nomor baris, mengikuti praktik error reporting compiler modern seperti Rustc atau Clang.

LAMPIRAN

Link Repository *Github* : <https://github.com/road2qnt/ABS-Tubes-IF2224-2026>

Pembagian Tugas

Nama	Pembagian Tugas	Kontribusi
Ega Luthfi Rais	Inisiasi awal code, Implementasi, dan debugging.	33.3%
Salman Faiz Assidqi	-	0%
Ahmad Fauzan Putra	Implementasi, debugging, dan Laporan	33.3%
Leonardus Brain Fatolosja	Implementasi, Debugging, dan Laporan	33.3%

REFERENSI

- GeeksforGeeks. (2026). Semantic Analysis in Compiler Design. Diakses dari <https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>
- GeeksforGeeks. (2026). Symbol Table in Compiler. Diakses dari <https://www.geeksforgeeks.org/compiler-design/symbol-table-compiler/>
- GeeksforGeeks. (2025). Type Checking in Compiler Design. Diakses dari <https://www.geeksforgeeks.org/compiler-design/type-checking-in-compiler-design/>
- GeeksforGeeks. (2025). Syntax Directed Translation in Compiler Design. Diakses dari <https://www.geeksforgeeks.org/compiler-design/syntax-directed-translation-in-compiler-design/>
- GeeksforGeeks. (2025). Intermediate Code Generation in Compiler Design. Diakses dari <https://www.geeksforgeeks.org/compiler-design/intermediate-code-generation-in-compiler-design/>
- Lab IRK(2026). Spesifikasi Milestone 3. Diakses dari [Spesifikasi Milestone 3 - Tubes IF2224 TBFO](#)